

13

External effects and I/O

In this chapter, we'll take what we've learned so far about monads and algebraic data types and extend it to handle *external effects* like reading from databases and writing to files. We'll develop a monad for I/O, aptly called `IO`, that will allow us to handle such external effects in a purely functional way.

We'll make an important distinction in this chapter between *effects* and *side effects*. The `IO` monad provides a straightforward way of embedding *imperative programming with I/O effects* in a pure program while preserving referential transparency. It clearly separates *effectful* code—code that needs to have some effect on the outside world—from the rest of our program.

This will also illustrate a key technique for dealing with external effects—using pure functions to compute a *description* of an effectful computation, which is then executed by a separate *interpreter* that actually performs those effects. Essentially we're crafting an embedded domain-specific language (EDSL) for imperative programming. This is a powerful technique that we'll use throughout the rest of part 4. Our goal is to equip you with the skills needed to craft your own EDSLs for describing effectful programs.

13.1 **Factoring effects**

We'll work our way up to the `IO` monad by first considering a simple example of a program with side effects.

Listing 13.1 Program with side effects

```
case class Player(name: String, score: Int)

def contest(p1: Player, p2: Player): Unit =
  if (p1.score > p2.score)
```

```

println(s"${p1.name} is the winner!")
else if (p2.score > p1.score)
  println(s"${p2.name} is the winner!")
else
  println("It's a draw.")

```

The contest function couples the I/O code for displaying the result to the pure logic for computing the winner. We can factor the logic into its own pure function, `winner`:

```

def winner(p1: Player, p2: Player): Option[Player] =
  if (p1.score > p2.score) Some(p1)
  else if (p1.score < p2.score) Some(p2)
  else None

def contest(p1: Player, p2: Player): Unit = winner(p1, p2) match {
  case Some(Player(name, _)) => println(s"$name is the winner!")
  case None => println("It's a draw.")
}

```

Contains the logic for computing the winner, or the fact that there is a draw

Has the responsibility of declaring the winner on the console

It is *always* possible to factor an impure procedure into a pure “core” function and two procedures with side effects: one that supplies the pure function’s input and one that does something with the pure function’s output. In listing 13.1, we factored the pure function `winner` out of `contest`. Conceptually, `contest` had two responsibilities—it was *computing* the result of the contest, and it was *displaying* the result that was computed. With the refactored code, `winner` has a single responsibility: to compute the winner. The `contest` method retains the responsibility of printing the result of `winner` to the console.

We can refactor this even further. The `contest` function *still* has two responsibilities: it’s computing which message to display and then printing that message to the console. We could factor out a pure function here as well, which might be beneficial if we later decide to display the result in some sort of UI or write it to a file instead. Let’s perform this refactoring now:

```

def winnerMsg(p: Option[Player]): String = p map {
  case Player(name, _) => s"$name is the winner!"
} getOrElse "It's a draw."

def contest(p1: Player, p2: Player): Unit =
  println(winnerMsg(winner(p1, p2)))

```

Has the responsibility of determining which message is appropriate

Has the responsibility of printing the message to the console

Note how the side effect, `println`, is now only in the *outermost* layer of the program, and what’s inside the call to `println` is a pure expression.

This might seem like a simplistic example, but the same principle applies in larger, more complex programs, and we hope you can see how this sort of refactoring is quite natural. We aren’t changing what our program does, just the internal details of how it’s factored into smaller functions. The insight here is that *inside every function with side effects is a pure function waiting to get out*.